Assignment – 16.1

2303A51655

Batch No :- 29

Task 1 – Student Information System Schema

PROMPT :

--create tables Students, Courses, Enrollments, primary keys, foreign keys, and relationships

CODE AND OUTPUT :

AI-16.1.sql > ...

```sql
--create tables Students, Courses, Enrollments, primary keys, foreign keys, and relationsh
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100)
);
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    course_code VARCHAR(10)
);
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
--insert sample data into Students, Courses, Enrollments tables
INSERT INTO Students (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'johndoe@gmail.com'),
(2, 'Jane', 'Smith', 'janesmith@gmail.com'),
(3, 'Michael', 'Johnson', 'michaeljhonson@gmail.com');
INSERT INTO Courses (course_id, course_name, course_code) VALUES
(1, 'Introduction to Computer Science', 'CS101'),
(2, 'Data Structures and Algorithms', 'CS102'),
(3, 'Database Systems', 'CS103');
INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date) VALUES
(1, 1, 1, '2024-01-15'),
(2, 1, 2, '2024-01-16'),
(3, 2, 1, '2024-01-17'),
(4, 3, 3, '2024-01-18');
--query to retrieve all courses enrolled by a student
SELECT s.first_name, s.last_name, c.course_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE s.student_id = 1;
--query to count number of students in each course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 2 of 2

first_name	last_name	course_name
John	Doe	Introduction to Computer Science
John	Doe	Data Structures and Algorithms

SQL ▾ 1 / 1 1 - 3 of 3

course_name	student_count
Data Structures and Algorithms	1
Database Systems	1
Introduction to Computer Science	2

JUSTIFICATION :

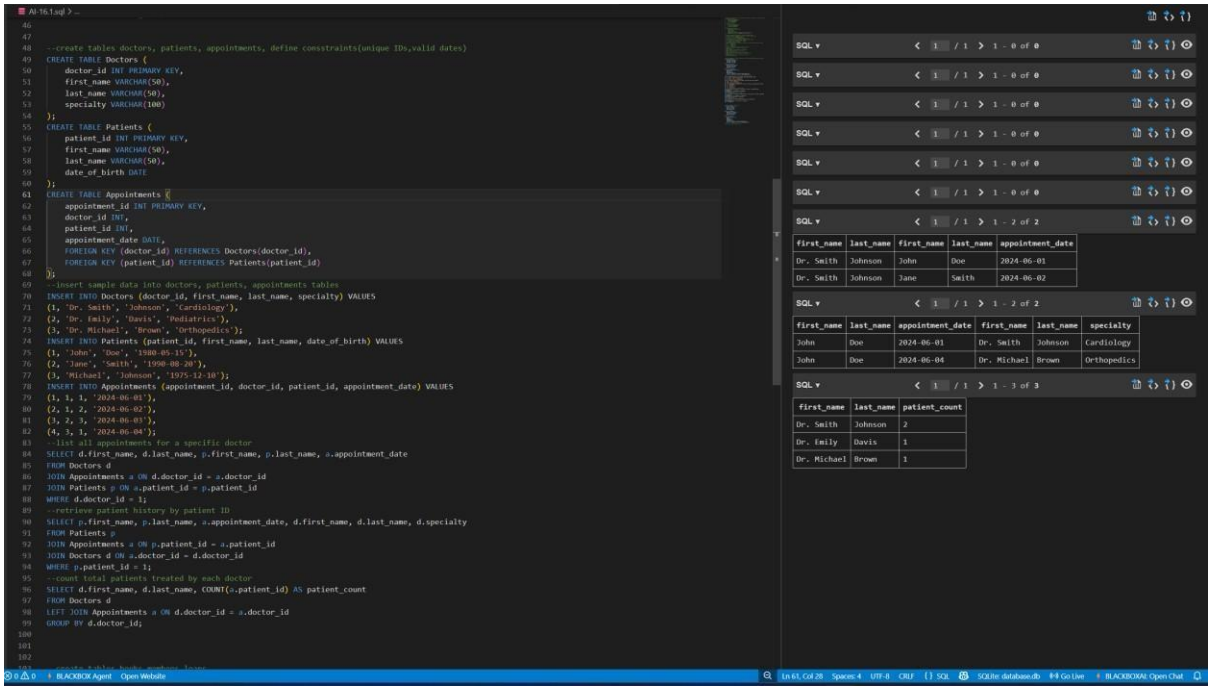
This task helped me understand how student and course data are stored in a database. I learned how to connect tables using primary key and foreign key. It also improved my knowledge of writing SQL queries to get useful information.

Task 2 – Hospital Management Database

PROMPT :

--create tables doctors, patients, appointments, define consstraints(unique IDs,valid dates)

CODE AND OUTPUT :



```sql
--create tables doctors, patients, appointments, define consstraints(unique IDs,valid dates)
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    specialty VARCHAR(100)
);
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    date_of_birth DATE
);
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
--insert sample data into doctors, patients, appointments tables
INSERT INTO Doctors (doctor_id, first_name, last_name, specialty) VALUES
(1, 'Dr. Smith', 'Johnson', 'Cardiology'),
(2, 'Dr. Emily', 'Davis', 'Pediatrics'),
(3, 'Dr. Michael', 'Brown', 'Orthopedics');
INSERT INTO Patients (patient_id, first_name, last_name, date_of_birth) VALUES
(1, 'John', 'Doe', '1980-05-15'),
(2, 'Jane', 'Smith', '1990-08-20'),
(3, 'Michael', 'Johnson', '1975-12-10');
INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
(1, 1, 1, '2024-06-01'),
(2, 1, 2, '2024-06-02'),
(3, 2, 3, '2024-06-03'),
(4, 3, 1, '2024-06-04');
--list all appointments for a specific doctor
SELECT d.first_name, d.last_name, p.first_name, p.last_name, a.appointment_date
FROM Doctors d
JOIN Appointments a ON d.doctor_id = a.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE d.doctor_id = 1;
--retrieve patient history by patient ID
SELECT p.first_name, p.last_name, a.appointment_date, d.first_name, d.last_name, d.specialty
FROM Patients p
JOIN Appointments a ON p.patient_id = a.patient_id
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE p.patient_id = 1;
--count total patients treated by each doctor
SELECT d.first_name, d.last_name, COUNT(a.patient_id) AS patient_count
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
```

first_name	last_name	first_name	last_name	appointment_date
Dr. Smith	Johnson	John	Doe	2024-06-01
Dr. Smith	Johnson	Jane	Smith	2024-06-02

first_name	last_name	appointment_date	first_name	last_name	specialty
John	Doe	2024-06-01	Dr. Smith	Johnson	Cardiology
John	Doe	2024-06-04	Dr. Michael	Brown	Orthopedics

first_name	last_name	patient_count
Dr. Smith	Johnson	2
Dr. Emily	Davis	1
Dr. Michael	Brown	1

JUSTIFICATION :

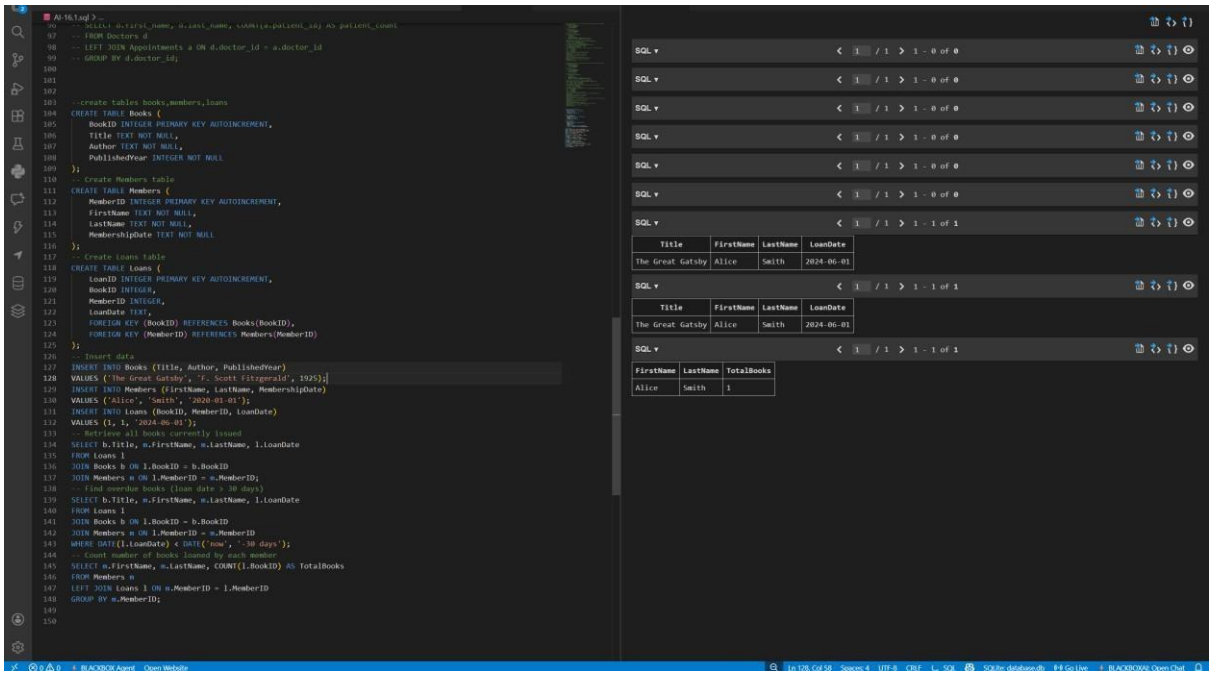
In this task, I understood how databases are used in hospitals to manage doctors, patients, and appointments. I learned about using constraints to avoid wrong data. It also helped me practice queries for real-life situations.

Task 3 – Library Database

PROMPT :

--create tables books,members,loans

CODE AND OUTPUT :



```sql
--create tables books,members,loans
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER NOT NULL
);
-- Create Members table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    MembershipDate TEXT NOT NULL
);
-- Create Loans table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    BookID INTEGER,
    MemberID INTEGER,
    LoanDate TEXT,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Insert data
INSERT INTO Books (Title, Author, PublishedYear)
VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 1925);
INSERT INTO Members (FirstName, LastName, MembershipDate)
VALUES ('Alice', 'Smith', '2020-01-01');
INSERT INTO Loans (BookID, MemberID, LoanDate)
VALUES (1, 1, '2024-06-01');
-- Retrieve all books currently issued
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID;
-- Find overdue books (loan date > 30 days)
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE DATE(l.LoanDate) < DATE('now', '-30 days');
-- Count number of books loaned by each member
SELECT m.FirstName, m.LastName, COUNT(l.BookID) AS TotalBooks
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Title	FirstName	LastName	LoanDate
The Great Gatsby	Alice	Smith	2024-06-01

Title	FirstName	LastName	LoanDate
The Great Gatsby	Alice	Smith	2024-06-01

FirstName	LastName	TotalBooks
Alice	Smith	1

JUSTIFICATION :

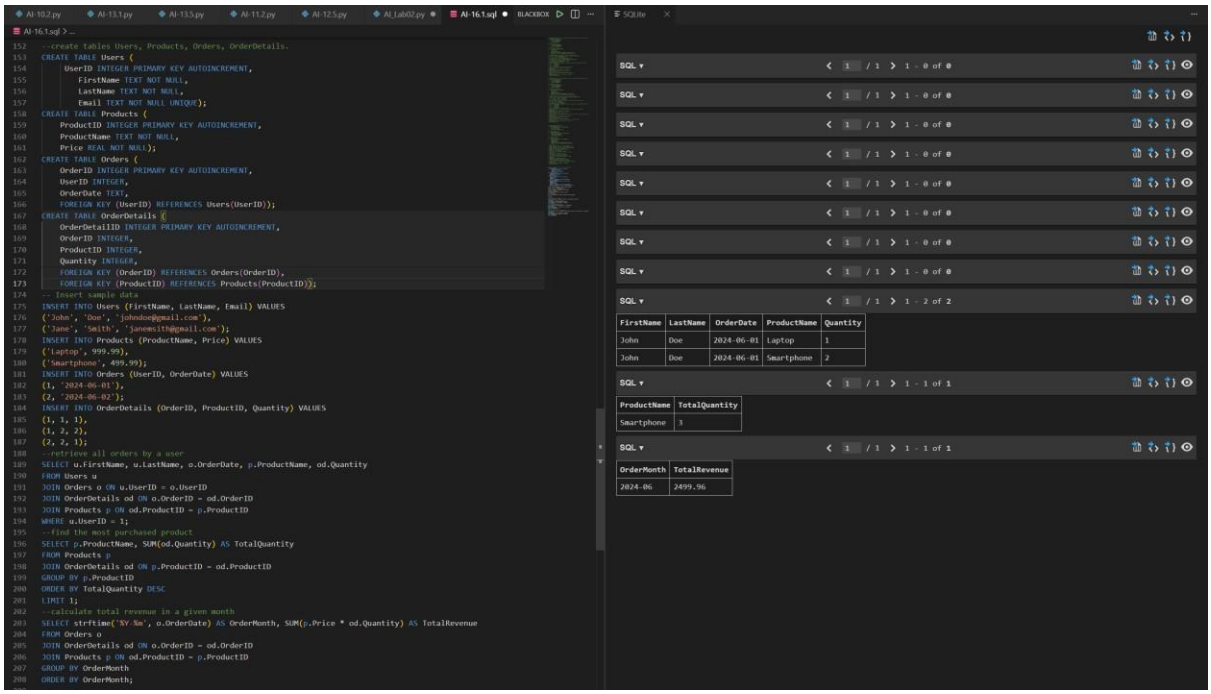
This task helped me learn how a library system works using a database. I understood how to track books and members. I also learned how indexing improves performance and how to find overdue books using queries.

Task 4 – Real-Time Application: Online Shopping Database

PROMPT :

--create tables Users, Products, Orders, OrderDetails.

CODE AND OUTPUT :



```sql
--create tables Users, Products, Orders, OrderDetails.
CREATE TABLE Users (
    UserID INTEGER PRIMARY KEY AUTOINCREMENT,
        FirstName TEXT NOT NULL,
        LastName TEXT NOT NULL,
        Email TEXT NOT NULL UNIQUE);
CREATE TABLE Products (
    ProductID INTEGER PRIMARY KEY AUTOINCREMENT,
    ProductName TEXT NOT NULL,
    Price REAL NOT NULL);
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID INTEGER,
    OrderDate TEXT,
    FOREIGN KEY (UserID) REFERENCES Users(UserID));
CREATE TABLE OrderDetails (
    OrderDetailID INTEGER PRIMARY KEY AUTOINCREMENT,
    OrderID INTEGER,
    ProductID INTEGER,
    Quantity INTEGER,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID));
-- Insert sample data
INSERT INTO Users (FirstName, LastName, Email) VALUES
('John', 'Doe', 'johndoe@gmail.com'),
('Jane', 'Smith', 'janesmith@gmail.com');
INSERT INTO Products (ProductName, Price) VALUES
('laptop', 999.99),
('Smartphone', 499.99);
INSERT INTO Orders (UserID, OrderDate) VALUES
(1, '2024-06-01'),
(2, '2024-06-02');
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 2, 1);
--retrieve all orders by a user
SELECT u.FirstName, u.LastName, o.OrderDate, p.ProductName, od.Quantity
FROM Users u
JOIN Orders o ON u.UserID = o.UserID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE u.UserID = 1;
--find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
FROM Products p
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID
ORDER BY TotalQuantity DESC
LIMIT 1;
--calculate total revenue in a given month
SELECT strftime('%Y-%m', o.OrderDate) AS OrderMonth, SUM(p.Price * od.Quantity) AS TotalRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY OrderMonth
ORDER BY OrderMonth;
```

FirstName	LastName	OrderDate	ProductName	Quantity
John	Doe	2024-06-01	Laptop	1
John	Doe	2024-06-01	Smartphone	2

ProductName	TotalQuantity
Smartphone	3

OrderMonth	TotalRevenue
2024-06	2499.96

JUSTIFICATION :

This task gave me an idea about how e-commerce websites store data. I learned how different tables like users, products, and orders are connected. Writing queries like total revenue helped me understand business analysis.

Task 5 – University Examination Database

PROMPT :

--create tables Student,Subjects,Exams,Results.primary keys, foreign keys, referential integrity constraints

CODE AND OUTPUT :

```sql
--create tables Student,Subjects,Exams,Results.primary keys, foreign keys, referential integrity constraints
CREATE TABLE Student (
    StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    DateOfBirth DATE NOT NULL
);
CREATE TABLE Subjects (
    SubjectID INTEGER PRIMARY KEY AUTOINCREMENT,
    SubjectName TEXT NOT NULL
);
CREATE TABLE Exams (
    ExamID INTEGER PRIMARY KEY AUTOINCREMENT,
    SubjectID INTEGER,
    ExamDate DATE NOT NULL,
    FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID)
);
CREATE TABLE Results (
    ResultID INTEGER PRIMARY KEY AUTOINCREMENT,
    StudentID INTEGER,
    ExamID INTEGER,
    Score INTEGER NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (ExamID) REFERENCES Exams(ExamID)
);
--insert sample data into Student, Subjects, Exams, Results tables
INSERT INTO Student (FirstName, LastName, DateOfBirth) VALUES
('John', 'Doe', '2000-01-01'),
('Jane', 'Smith', '2001-02-02');
INSERT INTO Subjects (SubjectName) VALUES
('Mathematics'),
('Science');
INSERT INTO Exams (SubjectID, ExamDate) VALUES
(1, '2024-06-01'),
(2, '2024-06-02');
INSERT INTO Results (StudentID, ExamID, Score) VALUES
(1, 1, 85),
(1, 2, 90),
(2, 1, 78),
(2, 2, 88);
--insert examination results for a student
--retrieve subject-wise marks for a student
SELECT s.FirstName, s.LastName, sub.SubjectName, r.Score
FROM Student s
JOIN Results r ON s.StudentID = r.StudentID
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects sub ON e.SubjectID = sub.SubjectID
WHERE s.StudentID = 1;
--calculate average marks for each subject
SELECT sub.SubjectName, AVG(r.Score) AS AverageScore
FROM Subjects sub
JOIN Exams e ON sub.SubjectID = e.SubjectID
JOIN Results r ON e.ExamID = r.ExamID
GROUP BY sub.SubjectID;
```

SQL 1 / 1 1 - 0 of 0

SQL 1 / 1 1 - 0 of 0

SQL 1 / 1 1 - 0 of 0

SQL 1 / 1 1 - 0 of 0

SQL 1 / 1 1 - 0 of 0

SQL 1 / 1 1 - 0 of 0

SQL 1 / 1 1 - 0 of 0

SQL 1 / 1 1 - 2 of 2

FirstName	LastName	SubjectName	Score
John	Doe	Mathematics	85
John	Doe	Science	90

SQL 1 / 1 1 - 2 of 2

SubjectName	AverageScore
Mathematics	81.5
Science	89.0

JUSTIFICATION :

This task helped me understand how exam data is managed. I learned how to store marks and calculate averages using SQL. It also improved my skills in using joins and aggregation functions.